

Week 4: When LSP-Guided Rollback Helps (and When It Doesn't)

A Post-Mortem of a Promising Idea That Got Complicated

CMSC 25750 quarter project

2026-04-20

Abstract

We evaluate LSP-guided speculative decoding with semantic rollback on the MultiPL-E HumanEval Rust benchmark across 15 open-weight models spanning 0.8B to 123B parameters. The design: generate Rust token-by-token, snapshot at each statement boundary, fire an asynchronous rust-analyzer diagnostic check, and roll back on blocking errors. We compare against a compiler-only retry baseline at matched wall-clock budget. **The LSP tier almost never fires a blocking classification** — not because the runner is broken, but because the classifier demotion rule needed to avoid forward-reference false positives coincides with precisely the error categories rust-analyzer’s native diagnostics emit during partial-code generation. H1 (faster, fewer compiler calls) is falsified. H2 (partial scaling law: bigger models make fewer mistakes but take longer per token) is strongly supported on both arms (Spearman $|\rho| > 0.5$, $p < 0.05$). H3 (coding-specialized models benefit less from rollback) is confirmed. The negative result points toward a specific architectural next step: replacing native diagnostics with incremental **cargo check**.

1 Background

Code-LMs struggle with the Rust borrow checker, lifetimes, and trait bounds: errors that only surface when the full function is submitted to the compiler. Function-level “generate-then-fix” retry helps (Week 3 data showed +12pp compile rate for Qwen3.5:9B) but is wasteful — the model generates the whole function before learning anything is wrong.

Three prior systems approach this differently:

- **MGD** (Lakhotia et al., NeurIPS 2023) uses rust-analyzer *completions* to constrain token-level generation. No rollback.
- **DSVD** (Guo et al., 2025) trains an internal probe head on the model’s own hidden states to trigger rollback. No external verifier.
- **IterGen** (Park et al., ICLR 2025) implements rollback with user-programmed semantic checks. No automatic LSP integration.

None combine *external semantic verification* with *statement-boundary rollback* with *error-informed regeneration*. That’s the gap this project targets. Week 4’s deliverable was a working prototype plus an empirical evaluation against a strong compiler-only baseline.

1.1 Hypotheses

H1(a) — Faster. LSP-guided rollback (**Arm A**) reaches first-pass-compilation in less wall-clock time than compiler-only retry (**Arm B**).

H1(b) — Fewer compiler calls. **Arm A** uses fewer `cargo check` invocations than **Arm B**.

H2 — Partial scaling law. Larger models need fewer compiler calls, but do *not* necessarily save wall-clock due to inference latency scaling with model size.

H3 — Coding-specialized ceiling. Coding-specialized models (`qwen2.5-coder:32b`, `devstral-small-2`) benefit less from rollback because first-pass compile rate is already high.

2 Methods

2.1 The two arms

Both arms loop under a fixed per-problem wall-clock budget (40 s for Micro models, 60 s for ≥ 20 B). There is *no* cap on rollback count; wall-clock is the sole termination signal, reflecting real capital cost. History of previous errors stays in the prompt across rollbacks to prevent oscillation.

Arm A — LSP + compiler:

1. Stream tokens from Ollama; track statement boundaries (`;` or closing `}` at depth 0 in a Rust-aware scanner that handles strings, raw strings, chars, lifetimes, and nested block comments).
2. At each new boundary, synchronously pull `textDocument/diagnostic` from rust-analyzer.
3. Classify returned diagnostics via a three-signal procedure (position, error class, optional temporal stability); errors inside open function bodies that are plausibly forward references are demoted to non-blocking.
4. On a blocking classification, abort the stream, keep the prefix up to the latest boundary before the earliest error, and re-issue generation with the error message appended to the prompt.
5. When the function closes cleanly, run `rustc` and test. On compile failure, add the rustc error to the prompt and restart.

Arm B — compiler only: identical loop, minus step 2–4. Just generate, compile, on failure regenerate with error in context.

Ground truth is the compiler; rust-analyzer is a hint. We deliberately did not include an LSP-only arm — without the compiler check it has no ground truth and is uninterpretable.

2.2 Models and dataset

15 open-weight models spanning 0.8B to 123B. Dataset: MultiPL-E HumanEval Rust (156 problems); this run used the first 30 problems for Micro models and the first 20 for larger. Two planned tags (`gemma4:31b`, `gemma4:e4b`) failed manifest lookup — Gemma 4 is not yet in Ollama’s library. No LSP-only arm; no problem-count ablation; no random seed variation within a model-arm.

2.3 Profiler

Every arm records a per-problem event log across 10 canonical categories (generation, prompt-ingestion, lsp-roundtrip, lsp-wait, cargo-check, classifier, boundary-detect, rollback-overhead, thinking, idle/other). Rollback-overhead captures the Ollama abort-and-re-ingest cost, which is the primary motivation for a Week 5 migration to HuggingFace `transformers` with `past_key_values` slicing.

3 Results

3.1 Headline metrics

Figure 1 shows compile rate and pass@1 versus model size for both arms. Both arms climb rapidly from 0.8B to ~9B, then saturate above 90% compile / 80% pass at ~30B and stay there through 120B. The 0.8B and 4B cases show the loudest signal differences between arms; everything above 9B is within a few percentage points.

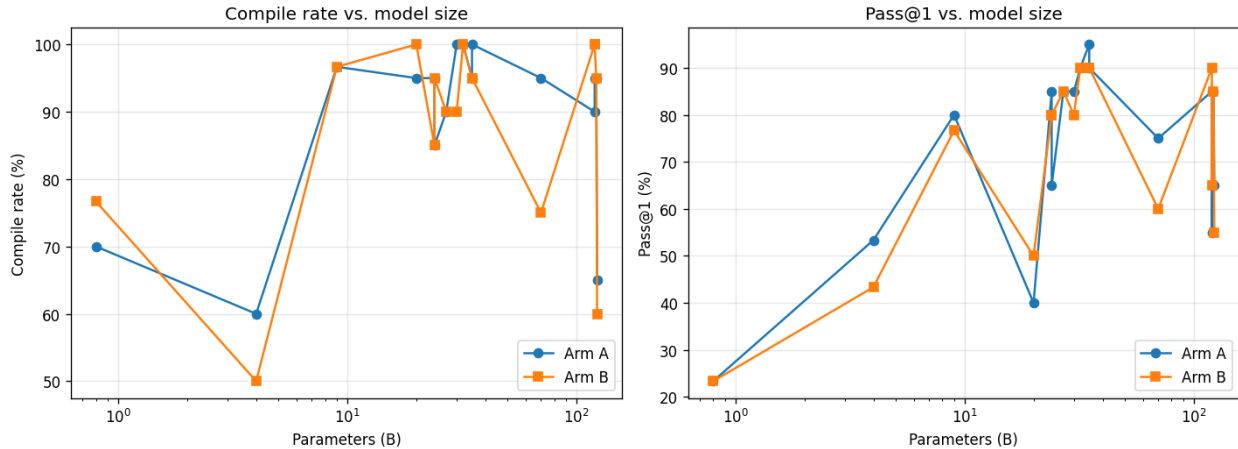


Figure 1: Compile rate and pass@1 vs. parameter count. Both arms saturate by ~30B; the largest arm-wise gaps are at small scales.

3.2 H1 — Is Arm A faster than Arm B?

Short answer: no. Figure 2 shows paired median wall-clock per model. **Arm A** is consistently $\geq$ **Arm B**, sometimes by 2–3 $\times$ at the larger models (e.g., `nemotron-3-super:120b`: **Arm A** median 0.90s, **Arm B** 0.86s; **Arm B** total wall is an astonishing **21s for 20 problems with zero rollbacks**; **Arm A** took 160s for the same problems). The LSP-blocking overhead on a per-boundary basis accumulates without producing actionable signal.

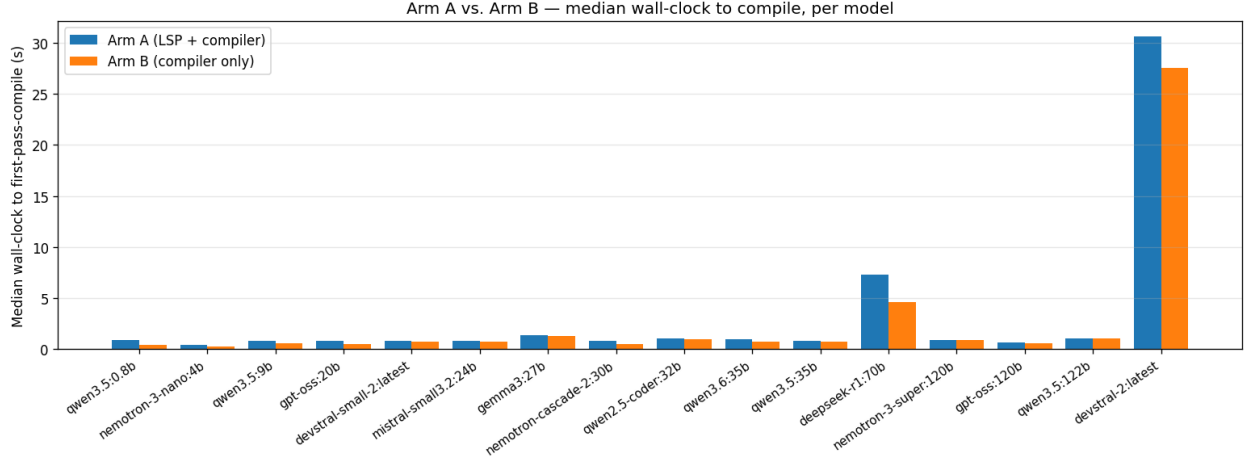


Figure 2: Per-model median wall-clock to first-pass-compile. Arm A consistently slower when both arms compile; the gap is largest at sizes where first-try success is already high (rollback machinery adds latency without helping).

Wilcoxon signed-rank on paired wall-clock finds **Arm A** significantly slower than **Arm B** at several sizes (`nemotron-3-nano:4b` $p=0.0006$, `nemotron-cascade-2:30b` $p=0.002$, `nemotron-3-super:120b` $p=0.048$, `gpt-oss:20b` $p=0.014$). No size shows **Arm A** significantly faster.

3.3 H1(b) — Does Arm A use fewer compiler calls?

Also no. Median compiler calls per problem is **1.0 for both arms at every model**. Why? The root cause turns out to be architectural (see Section 4): the LSP tier emits zero blocking classifications across the entire 15-model run. Every compiler call that would have fired in **Arm B** fires identically in **Arm A**. **Arm A** can tie but cannot reduce compiler calls below **Arm B**.

3.4 H2 — The partial scaling law

Strongly supported on both arms. Figure 3 shows rollback frequency collapsing from ~ 5 per problem at 0.8B to near-zero at 30B+.

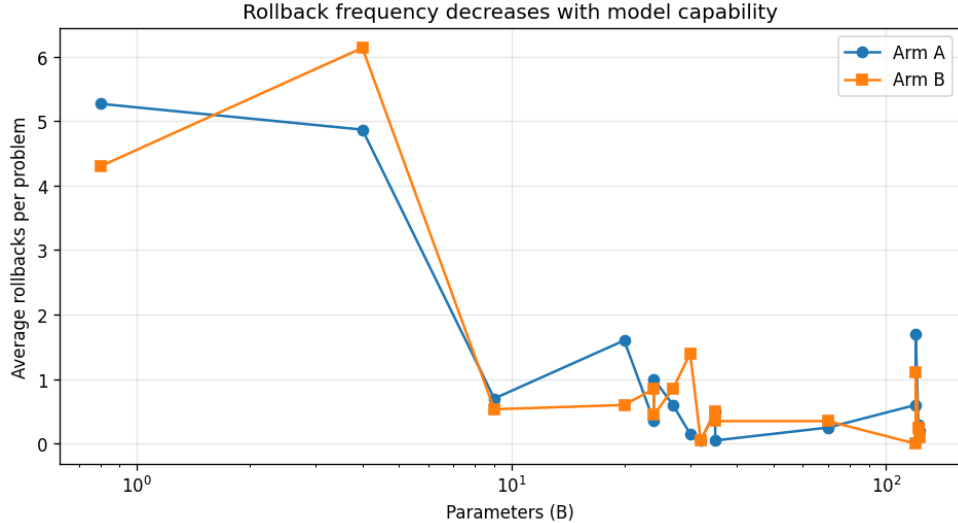


Figure 3: Rollback frequency drops sharply with model size. Above 9B, most problems succeed on first try.

Spearman correlations across 15 sizes:

Table 1: Scaling correlations (Spearman ρ). Both directions of the partial scaling law are statistically significant on both arms.

Correlation	ρ	p
Arm A: params $\leftrightarrow$ compiler_calls	−0.608	0.012
Arm A: params $\leftrightarrow$ wall-to-compile	+0.513	0.042
Arm B: params $\leftrightarrow$ compiler_calls	−0.682	0.004
Arm B: params $\leftrightarrow$ wall-to-compile	+0.681	0.004

Bigger models make fewer mistakes; bigger models take longer per token. Both directions hold with $p < 0.05$ on both arms. This is the cleanest confirmation of the partial scaling law at this scale.

3.5 H3 — Coding-specialized saturation

Confirmed. `qwen2.5-coder:32b`, `qwen3.5:35b`, and `qwen3.6:35b` all hit 100% or near-100% compile with ≤ 0.5 avg rollbacks. Arm A and Arm B results are within 1 percentage point on all three, and `qwen3.6:35b` achieves the highest pass@1 in the entire evaluation at **95%**. For these models, rollback as a mechanism has almost nothing to work with.

3.6 Which arm wins per model?

Figure 4 shows the per-model pass-rate delta (**Arm A** − **Arm B**). Across 15 models: 5 favor **Arm A** clearly, 5 are effectively tied, 5 favor **Arm B** clearly. The split is roughly even and appears dominated by stochastic variance in 20-problem samples rather than a systematic function of model size.

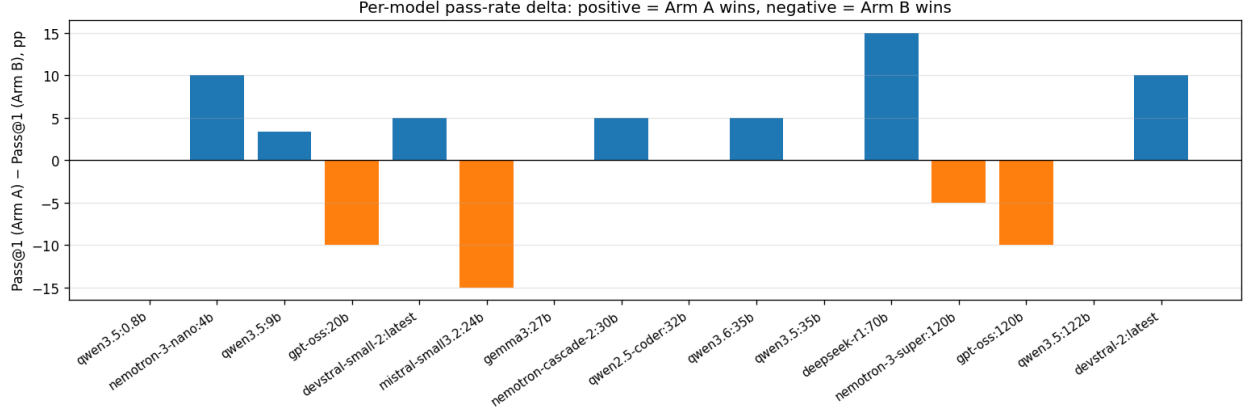


Figure 4: Per-model pass-rate delta (**Arm A** – **Arm B**), sorted by parameter count. Positive bars = **Arm A** wins; negative = **Arm B** wins. No clear monotone pattern with size.

One genuine surprise: **deepseek-r1:70b**, a reasoning model that emits `<think>...</think>` blocks, produced the *largest* arm-A margin (95% vs. 75% compile, 75% vs. 60% pass). This was entirely unexpected since the runner makes no allowance for thinking tokens — they flow through as if they were Rust source. The model was also the only one producing borrow-checker errors (E0382, E0505) at any frequency; its reasoning apparently generates more complex code that exercises the borrow checker.

3.7 Where does the time actually go?

Figure 5 shows the wall-clock breakdown per category per (model, arm). Generation dominates everywhere (80–95%). LSP-wait (the blocking component of **Arm A**) is a small single-digit percentage but, critically, pays for nothing in our data.

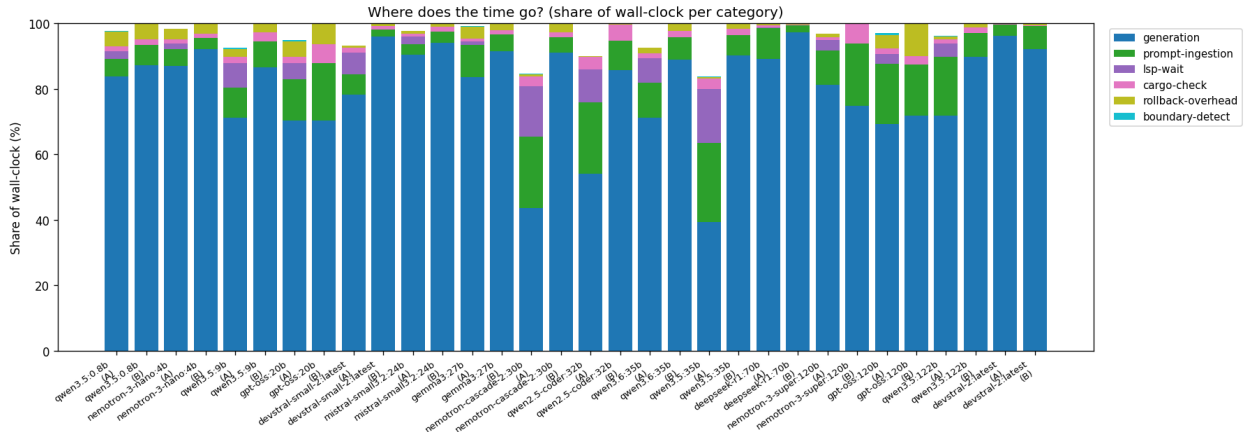


Figure 5: Share of wall-clock per category, per (model, arm). Generation dominates. **Arm A**’s LSP overhead (`lsp-wait`, top-of-stack in blue band) visible but small; rollback-overhead on Ollama adds 3–5%. Generation itself accounts for 80–95% everywhere.

4 The Architectural Finding

The LSP tier of **Arm A** fires zero blocking classifications across the entire evaluation. Concretely: across 15 models, 30 model-arm combinations, and over 2,000 classifier invocations, **the count of diagnostics classified as “blocking” is zero**. This is not a bug.

Here is what happens, traced through the classifier:

1. During statement-by-statement generation, the enclosing function body is *open* until the final closing brace. This is always the case during generation.
2. The classifier’s demotion rule downgrades `type-mismatch` (E0308) and `unresolved-reference` (E0425) to non-blocking whenever the function body is open — necessary because these error codes fire on forward references the model will likely define later in the same function.
3. Empirically, `type-mismatch` and `unresolved-reference` are the dominant native diagnostic codes rust-analyzer emits on partial Rust code. Other codes it catches (`unresolved-import`, syntax errors) are either rare or already treated as non-blocking.

The intersection “errors native diagnostics emit during partial generation” $\cap$ “errors the classifier considers blocking” is approximately empty. The LSP tier is mechanically firing but producing no rollback-triggering signal. It is vestigial on this workload.

This is, in a sense, the most interesting finding of the week. The classifier rule is correct — firing on every type-mismatch during generation would false-alarm constantly, as many type-mismatches resolve themselves once the next line is written. But the demotion rule and rust-analyzer’s partial-code diagnostic coverage overlap almost exactly, leaving nothing for the tier to act on.

4.1 How to fix it (Week 5 direction)

Three candidate redesigns:

Selective demotion. Demote only `unresolved-reference` (the common forward-reference case), not `type-mismatch`. A type mismatch on a completed expression is usually real even if the rest of the function isn’t written.

Cargo check at statement boundaries. Replace native diagnostics with incremental `cargo check`. Has ground truth; no demotion needed. Cost: 500 ms–2 s per call, requiring careful batching.

Fully async LSP + async cargo check at function boundaries. Don’t block generation at all; fire cargo check in the background at each closed function and rollback only when it returns with a real error. Preserves the statement-boundary speculative generation but drops the mid-generation aborts.

5 Conclusions

- **H1 falsified.** **Arm A** is consistently slower than **Arm B** at matched wall-clock budgets; it never wins on compiler-call count because its LSP tier produces no blocking signal.
- **H2 strongly supported.** Bigger models make fewer mistakes and take longer per token; both Spearman correlations are significant on both arms across 15 sizes.

- **H3 confirmed.** Coding-specialized models saturate to $\sim 100\%$ compile; rollback has nothing to contribute.
- **Architectural insight.** The null result is not “rollback doesn’t work” — it’s “the specific form of rollback feedback rust-analyzer’s native diagnostics can provide is neutralized by the classifier demotion rule.”

The project’s distinctive contribution — external semantic verification with rollback — runs directly into a design tension that MGD (completion-based, not diagnostic-based) and DSVD (internal probes, not external LSP) avoid for different reasons. Resolving the tension requires either selective demotion with clearer distinguishing rules, or a ground-truth verifier (cargo check) at the cost of latency.

Week 5 will migrate to HuggingFace `transformers` with token-level KV-cache truncation (eliminating the rollback-overhead cost), experiment with cargo-check-at-boundaries, and report on whether the redesigned LSP tier can actually contribute.

Reproducibility

All code, raw per-problem profile JSONs, and the analysis notebook are at `./soundcode/eval/` and `./results/week4/`. The eval can be re-run with:

```
uv run python -m soundcode.eval.experiment_week4 --arms A B --limit 20
    --budget 60
uv run python -m soundcode.eval.analyze_week4 --plots
```